

Ankita Kulkarni

351 Followers

About

Follow

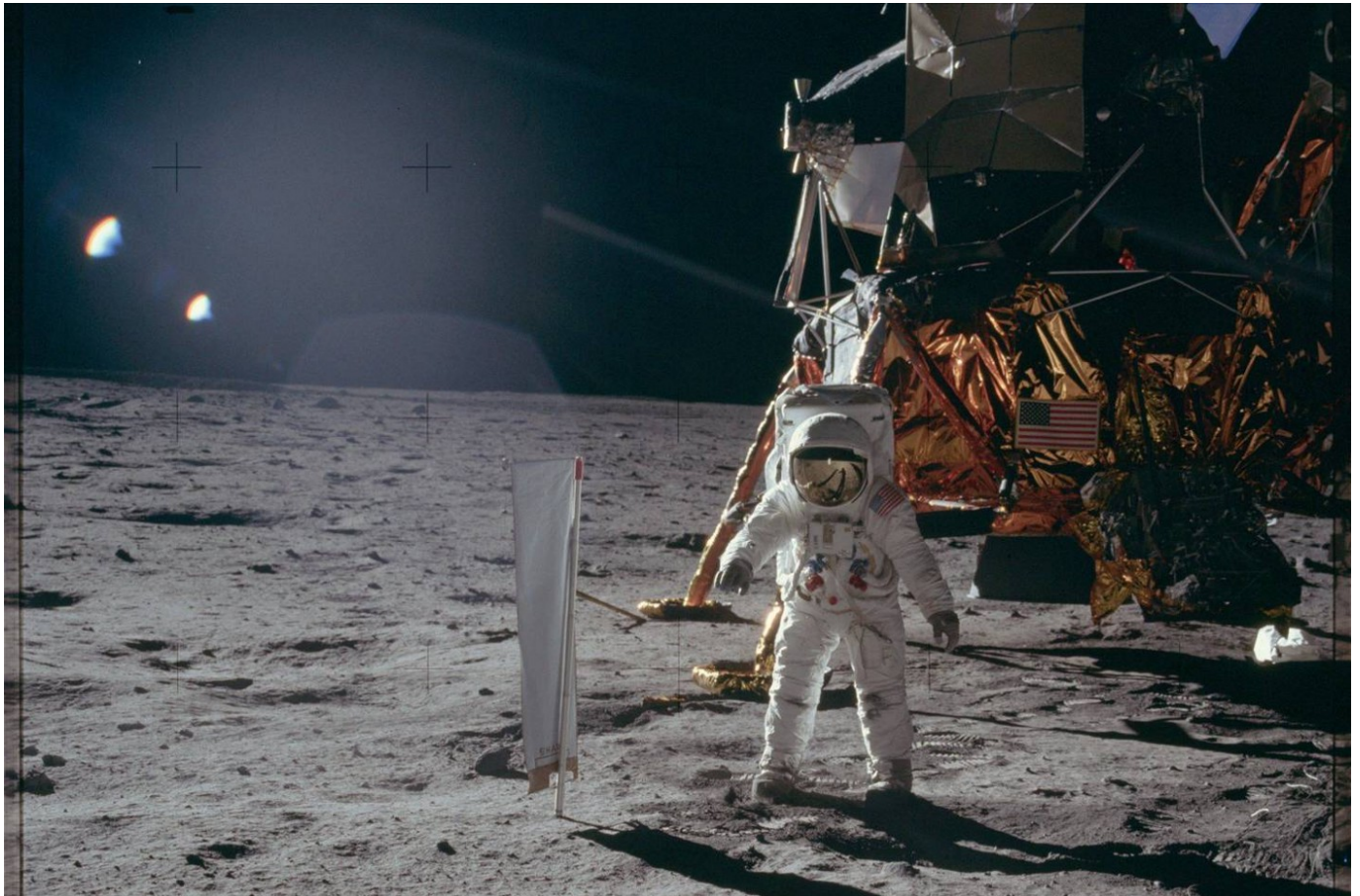


Oh Hello Apollo Client 🧡, Goodbye Redux! 🙋



Ankita Kulkarni Feb 1 · 10 min read

I know I got excited with the title there but it is kinda true 😂. In this blog post, I will cover why your switch to GQL and Apollo Client 3 should make you walk away from Redux. I will also talk about my journey from Redux -> Apollo Client.



I have had my share of doubts so in the past couple of projects, I was really skeptical of using *Apollo Client* as a state management solution. I love ❤️ Apollo and specifically the changes made in Apollo client 3 that changed my mind completely 🤗

Why I like Redux and what it is good at 💙

- Global state management solution where you have a good visual of your entire state
- You use actions to trigger state updates and asynchronous requests (love 📧 my boo: Redux saga 🔗)
- The entire ecosystem is amazing, you get *Redux* time travel too for debugging! 🕒
- You can use libraries like *Redux* selectors (another awesome library 🔗) to select data from the state and transform

which brings me to my next point... 📌

What is considered a good state management solution? ✅



1. My data is normalized (no dupes please 🙏)
2. Specific actions i.e. user logging in / routing should be able to trigger asynchronous requests 100
3. We want to transform the data so that our component is not huge and we can write tests!! 🍲
4. Lastly, visualize the store i.e. we can view our global state and debug easily 🌐

and I'm sure there are more but the above were the top ones in my list! 🏆

After I started using GQL ✨

- I didn't use Redux in the GQL project because we were using React Hooks and React Context and it felt repetitive because you can use `useReducer` and `useContext` where you can dispatch actions and update state
- Apollo Client exposes custom hooks 🚢 like `useQuery`, `useMutation` which automatically exposed `loading`, `success` and `error` states so I didn't need to trigger

3 different actions in my redux store i.e. `CART_REQUEST`, `CART_SUCCESS` and `CART_ERROR`. For example, here is a comparison ⚡

```
// Apollo Client
const { loading, data, error } = useQuery(GET_CART_QUERY)

// vs

// Redux Saga
import { call, put } from 'redux-saga/effects'

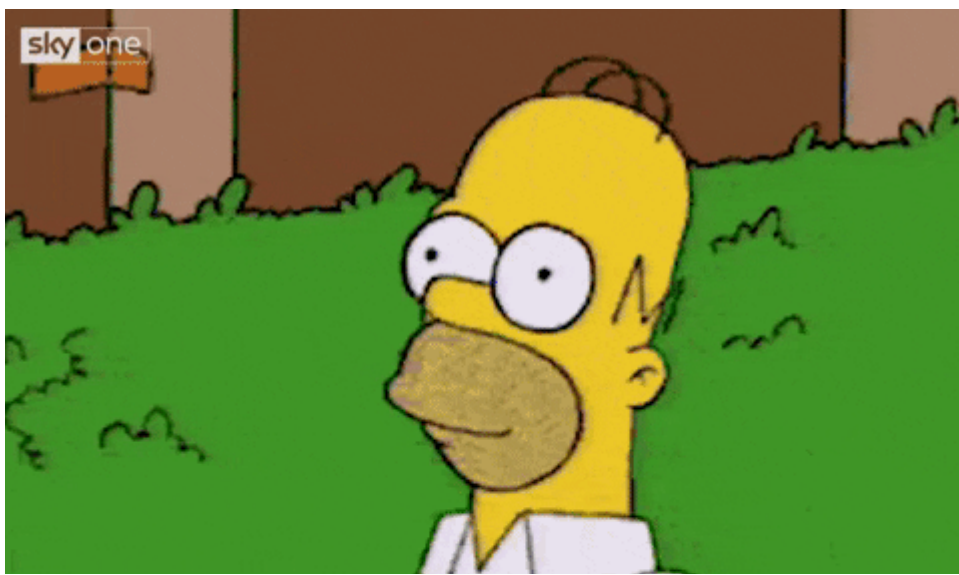
export function* fetchCartData(action) {
  try {
    const data = yield call(GET_CART_QUERY);
    yield put({ type: "CART_SUCCESS", { data, isLoading: false }});
  } catch (error) {
    yield put({ type: "CART_ERROR", { error, isLoading: false }});
  }
}

/*
  Listens for CART_REQUEST action getting dispatched and
  invokes fetchCartData
*/
function* watchFetchCartData() {
  yield takeEvery('CART_REQUEST', fetchCartData)
```

A lot of boilerplate code has reduced 🐱 You get the `loading`, `success`, and `error` states right from the `useQuery` and `useMutation` hook.

So what was missing? 🤔

Going back to the definition of a good state management library 🙌





- I really loved `useQuery` and `useMutation` custom hooks although I wasn't fully convinced to switch for state management completely as I really liked using *Redux selectors* that select data & we have the ability to transform it 😞
- In the meanwhile, I was using *React Context* instead of *Redux*
- I also didn't want to read the Apollo cache all the time
- At the time, there was no way to store values outside the cache
- I also wanted actions to trigger asynchronous requests like *Redux saga's* do 🚶
- On top of this, I found Apollo client cache really hard to read 😞

But with Apollo Client 3, they introduced Reactive Variables and local only fields that changed everything 💖

Apollo Client 3 gives us 2 really cool things 😎

1. Local only fields
2. Reactive Variables

They are fields that resolve on the client side itself by reading data from the cache if you want thus replacing the transformers in Redux. Let's take a look how that would work.

My data is normalized (no dupes please 🙏)

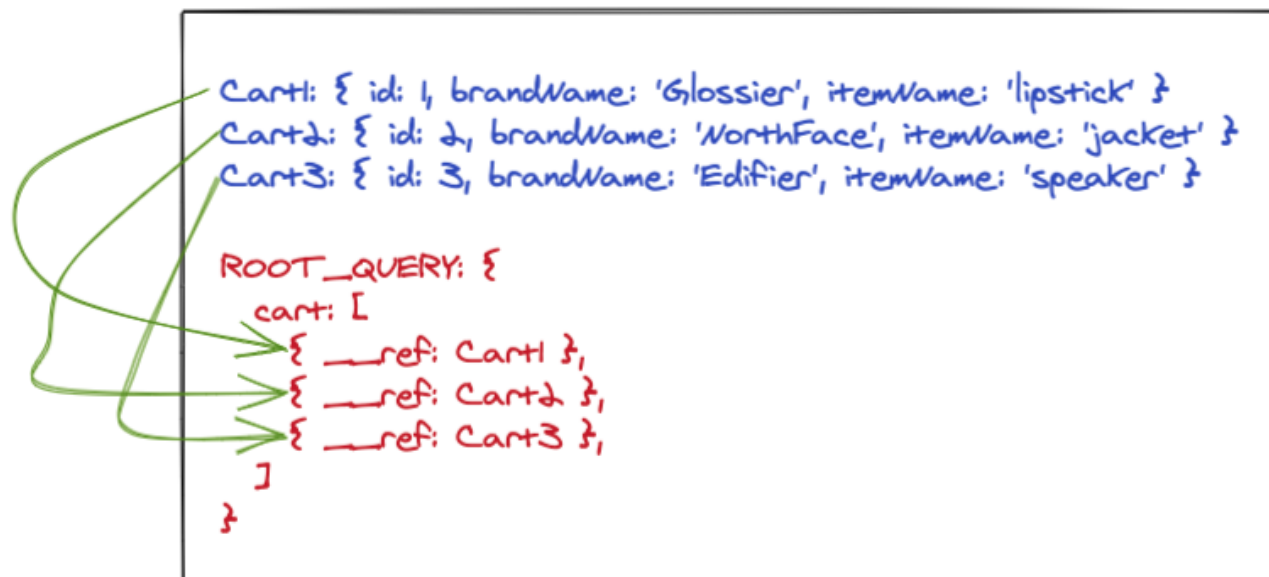




Apollo Client takes care of the heavy lifting for you 💪. You don't need to constantly dispatch actions to change state. With *redux*, we were really used to that and the benefit there is you have full control although do we really need full control? 😐

You are already using *GQL* ❤️ so everything is a graph 📊 and is stored in the graph i.e. you already have all your data in your cache then why add *Redux* on top to duplicate it? 🙄 You are going to add more overhead 🐼

Apollo Client automatically caches your data and normalizes new data in query responses and after mutation. Similar to what you would do in *Redux* where you would need to make sure that your data is normalized. If you are onboarding a new developer, it's hard because they also need to consider and learn how to do this on an architecture level which adds more overhead.



How the cache looks like

Apollo client stores data using **references** so it can be smart by looking it up easily using that reference as a key. Here is an [awesome blog post](#) 🔗 written by *Khalil Stemmler* on *Demystifying Apollo Cache* which you should read before switching to AC3 for state management. 🏆

Data transformations 😊

We want data transformations in an application so there is a clear separation of side-effect to transform layer. This way we can make sure that our component file is not huge and we can write tests for those transformers



We will use local only fields mainly for transforming data.

1. Local only fields 🌻

Local only fields is a way we can define client side fields on the GQL type that doesn't need to come from the server. You can resolve them locally on your frontend.

Let's say we have the following query for getting the user's cart ⚡

```
import { gql, useQuery } from '@apollo/client';

const GET_CART_QUERY = gql`
  query cart {
    id
    brandName
    itemName
  }
`
```

```
};  
  
// Invoking query inside the React Component  
const { loading, data, error } = useQuery(GET_CART_QUERY);
```

```
[  
  {  
    id: 1,  
    brandName: 'Glossier',  
    itemName: 'Lipstick',  
    price: {  
      numeric: 19.99,  
    },  
    outOfStock: false,  
    availableStock: 10  
  },  
  ....<more data>  
]
```

Here is how your cart query *data* object from the above query looks like 🙋

Let's say we have this user story, 🚀

As a user, I want to see if an item is out of stock or low stock based on the items in my cart. 🧑💻

Here is how your **React** component might look like for it without using the **Apollo client** **side** variable: 🚀 ⚡

```
const ViewStockText = () => {  
  const { loading, data, error } = useQuery(GET_CART_QUERY);  
  const [ outOrLowStockText, setOutOrLowStockText ] = useState('');  
  
  const getTextForLowOrOutOfStock = (data, itemsInCart) => {  
    const lowStock = 3;  
  
    const availableItemsToPurchase = data.availableStock - lowStock;  
  
    if (itemsInCart.length >= data.availableStock) {  
      return 'Out of stock'  
    } else if (data.itemsInCart.length >= availableItemsToPurchase) {  
      return 'Low stock'  
    }  
  }  
  
  useEffect(() => {
```

```

    if (data) {
      const text = getTextForLowOrOutOfStock(data);
      setOutOrLowStockText(text);
    }
  }, [data]);

  return <p>{outOrLowStockText}</p>
}

```

Typically in *Redux*, we would extract the logic of the function

`getTextForLowOrOutOfStock` outside using a redux selector. 

With AC3, you can achieve the above by **reading the cache** and adding the string for 'out of stock' and 'low stock' accordingly within your client itself.

OK But, how can we use local only fields? 🤔

We can create **local only fields** on the **Cart** type with the `@client` directive. 🎉 For example, ⚡ here `stockText` is the client side field.

```

import { gql, useQuery } from '@apollo/client';

const GET_CART_QUERY = gql`
  query cart {
    id
    brandName
    itemName
    stockText @client // Apollo client will resolve this client side
  }
`;

// Invoking query inside the React Component
const { loading, data, error } = useQuery(GET_CART_QUERY);

```

With the `@client` directive, Apollo client will look into the cache to resolve the field. It won't make a call over the network for that field because of the directive. Now `stockText` can be accessed anytime we declare a `Cart` type because it is a field on the `Cart` type.

```

import { InMemoryCache } from '@apollo/client'

const cache = new InMemoryCache({
  typePolicies: {
    Cart: {
      fields: {

```

```

    items: {
      stockText: {
        read(cart) {
          const lowStock = 3
          const availableItemsToPurchase = cart.availableStock - lowStock
          if (itemsInCart.length >= cart.availableStock) {
            return 'Out of stock'
          } else if (cart.itemsInCart.length >= availableItemsToPurchase) {
            return 'Low stock'
          }
        },
      },
    },
  },
},
},
},
})

export const client = new ApolloClient({
  cache,
})

```

Now we can directly access `stockText` in our React component by doing the following ⚡

```

import React from 'react';
import { gql, useQuery } from '@apollo/client';

const GET_CART_QUERY = gql`
  query cart {
    id
    brandName
    itemName
    stockText @client // Apollo client will resolve this client side
  }
`;

const ViewStockText = () => {
  const { loading, data, error } = useQuery(GET_CART_QUERY);
  return <p>{data?.stockText}</p>
}

```

2. Reactive Variables 🌸

We can also create custom client side values stored outside the cache known as Reactive Variables. Sometimes we just want to create a field outside of the type structure which can still be accessed through the *Apollo client* globally. For that, *Apollo client* gives us **Reactive variables**.

Modifying a reactive variable triggers an update of every active query that depends on that variable, as well an update of the react state associated with any variable values returned

from the `useReactiveVar` *React* hook.

Reactive variables don't update the cache but store the state information that we want to access at any point in our application. In *Redux*, we usually dispatch an action to store such a value in the store.

Let's say we have this user story, 📌

As a user, I want to view the number of items in my cart that are on sale. 100 🧑💻

For this you would have to read all the items in cart, check items that are on sale and count them. With Apollo client, you can achieve this by 📌 ⚡

```
import { InMemoryCache, makeVar } from "@apollo/client";

const cache = new InMemoryCache({
  typePolicies: {
    Query: {
      fields: {
        readNumberOf00SItems: {
          read(cart, { readField }) {
            const cartItems = readField('cart')?.data ?? [];
            const filteredItems = cartItems.filter(cartItem =>
cartItem.isOnSale);
            return filteredItems.length;
          }
        },
      },
    },
  },
});
```

You can do way more than this. You can also access existing fields (i.e.

`readNumberOf00SItems`) through other fields as well. 🙌

You can access the above `readNumberOf00SItems` via a query as well which gives you loading, data and error states:

```
import React from 'react';
import { gql, useQuery } from '@apollo/client';
```

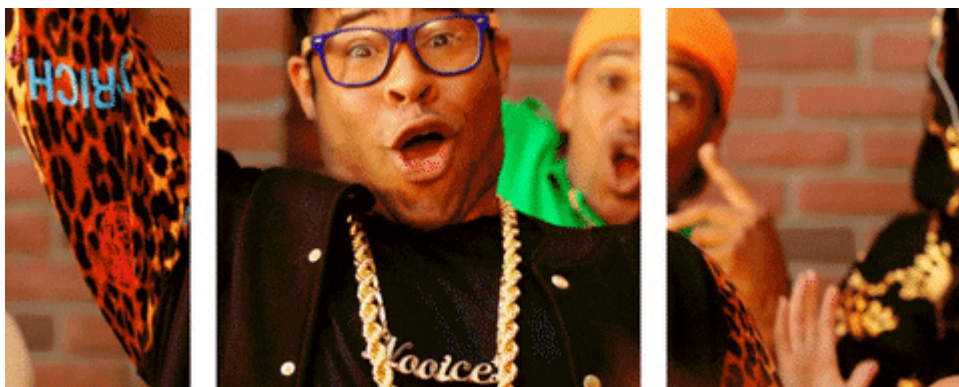
```
const GET_00S_ITEMS_QUERY = gql`
  query Get00SItems {
    readNumberOf00SItems @client
  }
`;

const ViewNumOf00SItems = () => {
  const { loading, data, error } = useQuery(GET_00S_ITEMS_QUERY);
  return <p>{data?.stockText}</p>
}
```

But wait, what is the difference between local only fields and reactive variables? 🤔

In a local only field, you create a new field on the type itself i.e. from our example, we created `stockText` on the `Cart` type i.e. you can't access `stockText` anywhere else.

But for reactive variables, you can access it **anywhere** you like and it isn't **restricted to a specific type**. Unlike the Apollo Client cache, reactive variables don't enforce data normalization, meaning **you can store data in any format you want**. 🍷



Specific actions should trigger asynchronous requests ▶▶

Once we do retrieve data or if the user wants to route based on certain information from the server, we might want to trigger asynchronous requests or rather specific actions that the user should take.

Let's say we have this user story, 🚪

As a user, I want to be taken to the sign in page if I'm not logged in else I want to see the app page

Here we want to track if the user is logged in or not and accordingly route the user. We can achieve this by creating a **reactive variable** for it.

Reactive variables are variables stored in the client and outside the cache but the components can also access their values directly. In the example below, `isUserLoggedIn` is a reactive variable that has been created using `makeVar` function. It invokes the function to check if there is a token in the browser *Cookies* 🍪. (In the real world, we will obviously check for token expiry as well 😊).

```
import { InMemoryCache, makeVar } from "@apollo/client";

const cache = new InMemoryCache({
  typePolicies: {
    Query: {
      fields: {
        readNumberOf00SItems: {
          read(cart, { readField }) {
            const cartItems = readField('cart')?.data ?? [];
            const filteredItems = cartItems.filter(cartItem =>
cartItem.isOnSale);
            return filteredItems.length;
          }
        },
        // reactive variable
        isUserLoggedIn: {
          read () {
            return isUserLoggedInVar();
          }
        },
      },
    },
  },
});

const isLoggedIn = () => Cookies.get('TOKEN')
// reactive variable
export const isUserLoggedInVar = makeVar(isLoggedIn());
```

Anything under fields is a **field policy**. A field policy is basically a contract between client and the function which dictates how that field is going to be resolved. We have a field policy to read the number of out of stock items and check if the user is logged in or not.

Next, in order to access this value within the component, we can do the following ⚡

```
import React from 'react';
import { useReactiveHook } from '@apollo/client';
```

```
import { isUserLoggedInVar } from './cache';
import Login from './login';
import App from './app';

const Main = () => {
  const isUserLoggedIn = useReactiveHook(isUserLoggedInVar);
  return isUserLoggedIn ? <App /> : <Login />;
}
```

The above will re-render whenever the value changes for `isUserLoggedInVar`

```
import React, { useEffect } from 'react';
import { useReactiveHook } from '@apollo/client';
import { isUserLoggedInVar } from './cache';

const App = () => {
  const isUserLoggedIn = useReactiveHook(isUserLoggedInVar);

  useEffect(() => {
    // trigger a async request here
  }, [isUserLoggedIn])

  return <div>Hello I am the main app</div>;
}
```

If you want to trigger an *API request* once the user has logged in, you can achieve this by listening to `isUserLoggedIn` in a `useEffect`. 🙌

Therefore, we can trigger async requests based on what's in the state.

But wait, can I update the value of the Reactive variable? 🤔

Yes you can! We can update the value of the reactive variable anywhere in our application, for example if we wanted to update the value of `isUserLoggedInVar` to `false` or anything else, we can! We just need to invoke the function `isUserLoggedInVar` directly!

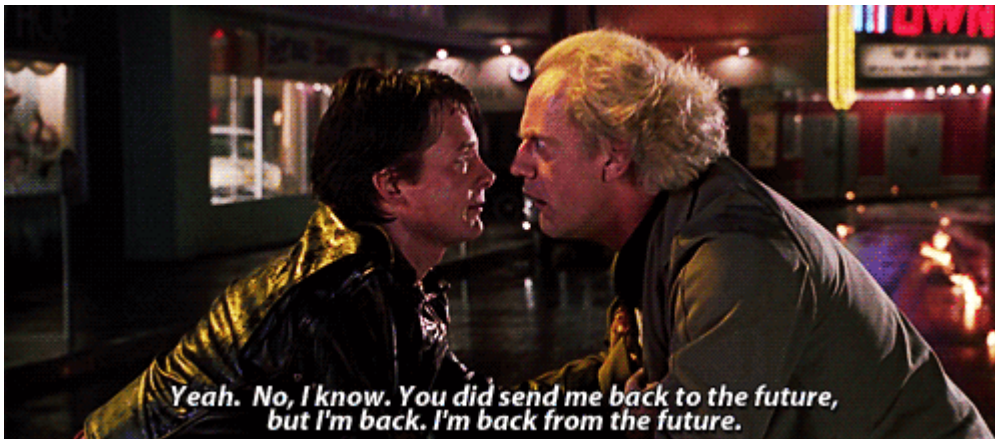
```
import React from 'react';
import { isUserLoggedInVar } from './cache';

const Logout = () => {
  const handleLogout = () => {
    isUserLoggedInVar(false); // will trigger component update
  }
}
```

```
return <button onClick={handleLogout}>Log out</button>;  
}
```

Visualize store / cache 🪄

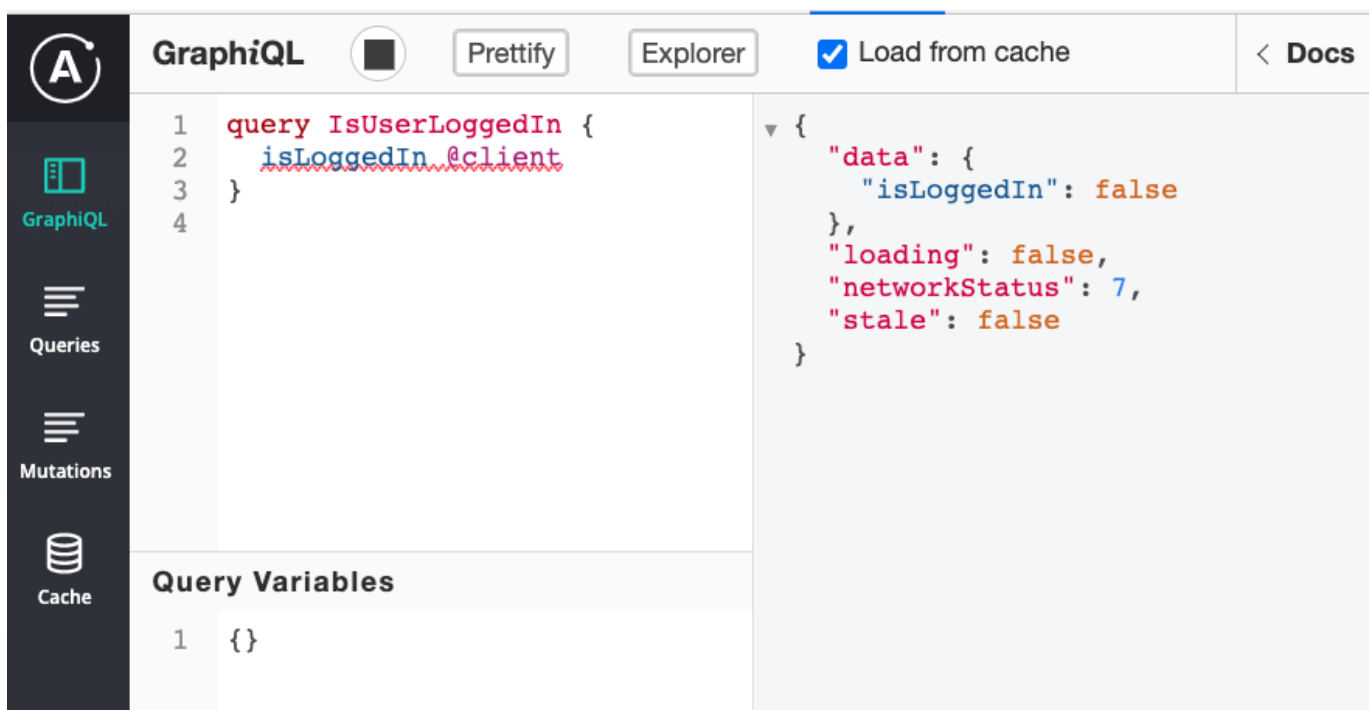
Lastly, visualize the store i.e. we can view our global state and debug easily



Just like *Redux developer tools*, *Apollo client* also have their developer tools, here is a [link](#).

🔗 Initially, I had some difficulty visualizing the cache as the Apollo developer tools are not as mature as Redux developer tools.

But after understanding how Apollo client stores data internally and how it optimizes it, things got a lot easier. I am able to visualize the cache. 😊



In the **Queries** and **Mutation** tab, you will see a list of Queries and Mutations executed in your application (just like Redux does). In the cache tab, you will see the entire cache

i.e. your Root query along with the cache references that got updated.

You can use **GraphiQL** to query anything (including Reactive variables) just like you would in the GQL playground. But **if you want to query Reactive variables**, make sure to check the checkbox “Load from cache”.

While I missed the time travel feature provided by Redux Dev Tools, once I learned what the cache looked like and how much it took care of, things became simpler to reason about. I would say lack of time travel is definitely a pain point of Apollo client dev tools overall 🤔

Lastly, keep an open mind ☀️

Keep an open mind :)

- The difference between *Redux* and *Apollo Client* is that you either take control and do everything on your own (like Redux) or let a mature library like Apollo Client handle that for you 🙌
- Don't get me wrong, I do love control 😂, but *Apollo client* is taking care of the bulk of the work for you so you can focus on the core of your application
- I kept comparing *Apollo client* to *Redux* 1:1 and although it was great to help me understand how my app would scale, this was also a reason I was holding back because now I have to unlearn what I have learned and trust that *Apollo client* will take care of it for you. 🙌

- When you are using Apollo client, it does feel redundant to use Redux on top of it as you are now keeping 2 copies of the same data i.e. Apollo client cache and Redux global store. 🤔
- The more you learn about the cache, the more you start to love it! ❤️

• • •

Thank you for making it so far, hope you found this post useful 100 and it helps you draw comparisons between Redux and *Apollo Client*. 🙌

GraphQL

React

Redux

Front End Development

JavaScript



[About](#) [Write](#) [Help](#) [Legal](#)

Get the Medium app

